Develando al Leviatán: Proceso ágil basado en Scrum para gestionar la deuda social en el desarrollo de software ágil

Universidad del Cauca

Luis Miguel Romero Vivas
Manuel Santiago Córdoba Galíndez

Universidad del Cauca
Facultad de Ingeniería Electrónica y Telecomunicaciones
Departamento de Sistemas
Grupo de investigación en tecnologías de la información – GTI
Línea de investigación ingeniería de software aplicada
Popayán, marzo de 2026

Develando al Leviatán: Proceso ágil basado en Scrum para gestionar la deuda social en el desarrollo de software ágil

Luis Miguel Romero Vivas
Manuel Santiago Córdoba Galíndez

Trabajo de investigación para optar al título de Ingenieros de Sistemas

Director: *César Jesús Pardo Calvache, PhD. MS.c.*

Universidad del Cauca
Facultad de Ingeniería Electrónica y Telecomunicaciones
Departamento de Sistemas
Grupo de investigación en tecnologías de la información – GTI
Línea de investigación ingeniería de software aplicada
Popayán, marzo de 2026

AGRADECIMIENTOS

A mi familia, por su paciencia, apoyo y amor incondicional durante toda la carrera universitaria.

A mi compañero de tesis, Manuel Santiago Córdoba Galíndez, por su colaboración, apoyo y amistad durante el desarrollo de este trabajo de grado.

A nuestro director de tesis, el profesor PhD. MS.c. César Jesús Pardo Calvache, por su apoyo profesional y personal, su paciencia y motivación durante todo el desarrollo de este trabajo de grado.

A mis amigos que me brindaron su apoyo, paciencia y comprensión durante este proceso.

Luis Miguel Romero Vivas
Pereira, 2026

AGRADECIMIENTOS

A mi familia, por el respaldo constante, la comprensión y el cariño incondicional que me acompañaron a lo largo de toda mi formación universitaria.

A mi compañero de tesis, Luis Miguel Romero Vivas, por su disposición, apoyo y compañerismo durante el desarrollo de este trabajo de grado.

A nuestro director de tesis, el profesor PhD. MS.c. César Jesús Pardo Calvache, por su orientación académica, calidad humana, paciencia y motivación a lo largo de todo el proceso investigativo.

A mis amigos, por su acompañamiento, comprensión y apoyo durante las distintas etapas de este camino.

Manuel Santiago Córdoba Galíndez
Bogotá, 2026

RESUMEN

La deuda social es un concepto que se refiere a las desigualdades y carencias que afectan a ciertos grupos dentro de una sociedad. En este contexto, el presente trabajo aborda la gestión de la deuda social mediante un enfoque ágil, proponiendo un proceso que permite identificar, priorizar y abordar las necesidades sociales de manera efectiva y eficiente.
Objetivo. El objetivo de este trabajo es desarrollar un proceso ágil para la gestión de la deuda social, que permita a las organizaciones y comunidades identificar y abordar las necesidades sociales de manera rápida y efectiva.
Métodos. Se utilizó un enfoque cualitativo basado en la revisión bibliográfica, diseño conceptual y validación mediante juicio de expertos para desarrollar el proceso ágil propuesto.
Resultados.
Conclusiones.
Trabajos futuros.
Palabras clave: Deuda social, gestión ágil, necesidades sociales, proceso ágil, comunidades, organizaciones.

ABSTRACT

The social debt is a concept that refers to the inequalities and needs that affect certain groups within a society. In this context, this work addresses the management of social debt through an agile approach, proposing a process that allows identifying, prioritizing and addressing social needs in an effective and efficient way.
Keywords: Social Debt, Agile, Scrum, Social Scrum.

ANEXO I: SISTEMA DE ESTIMACIÓN SOCIAL (SSP)

I.1 Sistema de estimación de items sociales (Social Story Points)

En Scrum tradicional, el Development Team estima el esfuerzo de los items técnicos usando Story Points (SP) con la escala Fibonacci (1, 2, 3, 5, 8, 13, 21). SocialScrum enfrenta un problema análogo: los items de deuda social también consumen tiempo y atención del equipo, pero no tienen una unidad de medida comparable. Esta sección introduce los Social Story Points (SSP), una escala de estimación donde 1 SSP $\approx$ 1 SP, permitiendo integrar items técnicos y sociales en un mismo Social Sprint Backlog.

I.1.1 Escala de estimación SSP

La escala SSP utiliza los mismos valores Fibonacci: 1, 2, 3, 5, 8 y 13. Valores superiores a 13 indican que el item debe descomponerse.

SSP	Duración	Tipo de intervención	Ejemplo
1–2	1–4 horas	Conversación facilitada, ajuste menor	Feedback rápido, norma simple
3–5	4 h–2 días	Workshop, mediación, retrospectiva profunda	Código de conducta, mediación bilateral
8–13	2 días–1 semana	Intervención organizacional, crisis	Cambio de liderazgo, burnout severo

Tabla 11.110: Escala de Social Story Points (SSP)

I.1.2 Metodología de estimación en 3 pasos

La estimación SSP sigue un proceso de tres pasos: identificar el *community smell*, evaluar severidad, y evaluar complejidad de intervención.

Paso	Criterio	Entrada	Salida
1	Identificar smell	Registro de Community Smells	Tipo específico confirmado
2	Evaluar severidad	Health Score y observación	Leve / Moderada / Severa / Crítica
3	Evaluar complejidad	Alcance, facilitación y sesiones	Baja (+0) / Media (+1) / Alta (+2)

Tabla 11.112: Proceso de estimación de SSP

I.1.2.1 Matriz de severidad y SSP base

Severidad	Indicadores	SSP base
Leve	Afecta 1–2 personas ocasionalmente; no impacta la entrega	1–2
Moderada	Afecta 3–5 personas; retrasos menores; tensión visible	3–5
Severa	Afecta al equipo entero; riesgo de rotación; calidad degradada	5–8
Crítica	Equipo disfuncional; rotación inminente; proyecto en riesgo	8–13

Tabla 11.114: Niveles de severidad y SSP base

I.1.2.2 Cálculo del SSP final

$$\text{SSP Final} = \text{SSP Base (severidad)} + \text{Ajuste (complejidad)} \tag{11.3}$$

Ejemplo: Conflicto Dev A-B (Severidad Moderada = 3 SSP base) + Mediación especializada requerida (Complejidad Alta = +2) → SSP Final = 5.

I.1.3 Tabla de referencia rápida

Community smell	Severidad típica	Intervención típica	SSP
Radio Silence	Moderada	Workshop y ajuste de proceso	3–5
Lone Wolf	Leve–Moderada	Mediación y reintegración	3–5
Conflicto interpersonal leve	Leve	Mediación bilateral simple	3
Conflicto interpersonal severo	Severa	Mediación intensiva y seguimiento	8
Burnout leve / moderado	Leve–Moderada	Conversación y ajuste de carga	2–5
Burnout severo	Severa–Crítica	Intervención de crisis	13
Inseguridad psicológica	Moderada–Severa	Workshop y retrospectiva facilitada	5–8
Knowledge hoarding	Moderada	Documentación y transferencia de conocimiento	3–5

Tabla 11.116: Referencia rápida de estimación SSP por *community smell*

I.1.4 Conversión SSP ↔ SP y gestión de capacidad

El principio fundamental es: **1 SSP ≈ 1 SP**. Ambos tipos de items pueden sumarse para calcular la capacidad total del Sprint.

Tipo	Ítems ejemplo 189	Puntos	Total
Técnico	Feature A (13 SP) + Feature B (8 SP) + Bug fix (5 SP)	26 SP	
Social	Mediación de conflicto (5 SSP) + Daily	8 SSP	

I.1.4.1 Mejores prácticas de estimación

- **Estimar conservadoramente**: Añadir 20 % de buffer en situaciones inciertas.
- **Descomponer items grandes**: Si un item supera 8 SSP, buscar cómo dividirlo.
- **Registrar real vs. estimado**: Mantener historial para calibración.
- **Un item = un community smell específico**: Evitar items vagos como "mejorar comunicación".

ANEXO J: CATÁLOGO DE ESTRATEGIAS DE MITIGACIÓN

Este anexo presenta un catálogo de estrategias de mitigación para diferentes categorías de *community smells*. Cada estrategia incluye una descripción, pasos recomendados y consideraciones clave para su implementación efectiva.

I.2 Estrategias de mitigación de community smells

Detectar un *community smell* es solo el primer paso. La detección sin acción genera frustración: el equipo sabe que hay un problema pero no ve mejora. Esta sección proporciona un catálogo de estrategias de mitigación basadas en la taxonomía de *community smells* de Caballero et al. **caballero2022communitysmells**, organizadas por categoría de smell. Herramientas como csDetector **almarimi2021csDetector** permiten detección automática de algunos de estos patrones a partir de metadatos de repositorios.
El objetivo no es eliminar todos los smells —algunos son síntomas de tensiones organizacionales que el equipo no controla— sino reducir su impacto en la capacidad del equipo de entregar valor.

I.2.1 Catálogo de estrategias por categoría

Para cada categoría de *community smell*, se describen problemas comunes, ejemplos representativos y tipos de intervención recomendados (ver Tabla 11.120).

Categoría	Problema	Smells representativos	Tipo de intervención
Comunicación	Información no fluye entre miembros o hacia afuera	*Radio Silence*, *Information Hoarding*, *Broadcast Storm*, *Missing Links*, *Synchronous Overload*	Rediseño de canales, protocolos y automatización
Conocimiento	Saber técnico concentrado o no compartido	*Truck Factor = 1*, *Knowledge Islands*, *Outdated Docs*, *Tribal Knowledge*, *Expert Bottleneck*	*Pair programming*, documentación y rotación
Relacional	Conflictos, exclusión y dinámicas tóxicas	*Lone Wolf*, *Prima Donna*, *Clique Formation*, *Unresolved Conflict*, *Toxic Leadership*	Mediación, facilitación y feedback directo
Organizacional	Estructura o procesos disfuncionales	*Organizational Silo*, *Meeting Hell*, *Unclear Ownership*, *Process Overhead*, *Unstable Team*	Escalamiento, negociación y reestructuración

Tabla 11.120: Categorías de *community smells*, ejemplos e intervenciones

I.2.1.1 Estrategias para *smells* de comunicación

Estas estrategias abordan problemas de flujo de información dentro del equipo y con *stakeholders* externos.

Smell	Síntomas	Estrategia de mitigación	SSP
Radio Silence	Sorpresas frecuentes; "nadie me dijo"; la información llega tarde	1) Identificar nodos centrales mediante SNA. 2) Crear un canal de broadcast para actualizaciones críticas. 3) Establecer *office hours* donde cualquiera pueda preguntar. 4) Rotar la responsabilidad de comunicar en la Daily.	3–5
Information Hoarding	Personas guardan información "por si acaso"; duplicación de esfuerzo	1) Workshop sobre beneficios de compartir. 2) Crear una wiki o Confluence con expectativa explícita de documentar decisiones. 3) Reconocer públicamente cuando alguien comparte información útil.	3–5
Broadcast Storm	Canales saturados; nadie lee mensajes; información importante se pierde	1) Auditar y consolidar canales existentes. 2) Establecer convenciones (por ejemplo, @here solo para urgencias). 3) Crear resúmenes diarios o semanales. 4) Eliminar canales redundantes.	2–3
Missing Links	Subgrupos no se comunican; integraciones tardías generan conflictos	1) Mapear la comunicación actual mediante SNA. 2) Crear rituales de sincronización entre subgrupos. 3) Asignar enlaces rotativos entre equipos. 4) Realizar retrospectivas conjuntas mensuales.	5–8
Synchronous Overload	Calendarios saturados; trabajo fragmentado; fatiga de videollamadas	1) Auditar reuniones y eliminar las innecesarias. 2) Establecer *no-meeting days*. 3) Convertir	3–5

Smell	Síntomas	Estrategia de mitigación	SSP
Truck Factor = 1	"Solo Juan sabe cómo funciona X"; bloqueos cuando Juan no está	1) Identificar componentes críticos con un solo experto. 2) *Pair programming* forzado: Juan + otro dev en cada tarea del componente. 3) Juan documenta la arquitectura básica (2–3 páginas). 4) Otro dev realiza un cambio pequeño de forma autónoma; Juan revisa.	5–8
Knowledge Islands	Subgrupos especializados no comparten conocimiento; la integración es dolorosa	1) *Tech talks* semanales rotativos (30 min). 2) *Code reviews* cruzados entre subgrupos. 3) Rotación temporal: un Sprint en otro subgrupo. 4) Documentación explícita de interfaces entre componentes.	5–8
Outdated Documentation	La documentación existe pero es incorrecta; nadie confía en ella	1) Auditar documentación: marcar obsoleta o eliminar. 2) Establecer *docs as code*: la documentación vive junto al código. 3) Incluir actualización de documentación en la *Definition of Done*. 4) Asignar un responsable de cada documentación crítica.	3–5
Tribal Knowledge	Conocimiento principalmente oral; el *onboarding* toma meses	1) Grabar sesiones de *onboarding* para reutilizarlas. 2) Crear un *playbook* de operaciones comunes. 3) *Pair programming* intensivo durante las primeras dos semanas. 4) Implementar un sistema de compañeros (*buddy system*) para nuevos miembros.	5–8
Expert Bottleneck	Una sola persona revisa todo; los PRs esperan días	1) El experto entrena a 2–3 personas como reviewers secundarios. 2) Establecer un SLA (Service Level Agreement): PRs simples no requieren aprobación del experto. 3) Crear checklists de revisión reutilizables. 4) Delegar gradualmente categorías de PRs.	5–8

Tabla 11.124: Estrategias para smells de conocimiento

I.2.1.3 Estrategias para smells relacionales

Estas estrategias se enfocan en mejorar las interacciones y relaciones dentro del equipo.

Smell	Síntomas	Estrategia de mitigación	SSP
Lone Wolf	Trabaja solo; no pide ni ofrece ayuda; PRs sin contexto	1) 1:1 para entender la motivación (preferencia o desconfianza). 2) Pair programming gradual (1 hora al día). 3) Asignar tareas que requieran colaboración. 4) Reconocer públicamente cuando colabora.	3–5
Prima Donna	Cree que sus ideas son superiores; no acepta feedback; bloquea el consenso	1) 1:1 para dar feedback específico sobre el impacto. 2) Establecer la norma de decisiones por consenso, no por imposición. 3) Documentar criterios objetivos para decisiones técnicas. 4) Si persiste, escalar al	5–8

Smell	Síntomas	Estrategia de mitigación	SSP
Organizational Silo	Equipos no se comunican; duplicación de trabajo; integraciones tardías	1) Identificar puntos de contacto necesarios. 2) Establecer embajadores entre equipos. 3) Reuniones de sincronización bisemanales. 4) Escalar a liderazgo si la estructura impide la colaboración.	5–8
Meeting Hell	Calendarios saturados por demandas externas al equipo	1) Auditar reuniones: quién las convoca y si son necesarias. 2) Negociar con *stakeholders* un representante único. 3) Establecer *focus time* protegido. 4) Escalar al Scrum Master para proteger al equipo.	3–5
Unclear Ownership	Preguntas frecuentes sobre responsables; decisiones postergadas	1) Mapear componentes y asignar responsables explícitos. 2) Documentar una matriz RACI para decisiones comunes. 3) Publicar el *ownership* en una wiki accesible. 4) Revisar y actualizar cada tres meses.	3–5
Process Overhead	Aprobaciones múltiples y burocracia que no agrega valor	1) Documentar tiempo perdido en procesos. 2) Proponer simplificación basada en evidencia. 3) Ejecutar un piloto con exención de un proceso durante dos Sprints. 4) Medir impacto y escalar resultados.	5–8
Unstable Team	Miembros prestados y rotación frecuente	1) Documentar el impacto en velocidad y calidad. 2) Escalar a liderazgo con datos. 3) Negociar un período mínimo de estabilidad. 4) Si no es posible cambiar, optimizar el onboarding.	5–8

Tabla 11.128: Estrategias para smells organizacionales

I.2.1.5 Smells que requieren escalamiento

Algunos smells no pueden resolverse a nivel de equipo. El Social Guide debe reconocer estos casos y escalar apropiadamente:

Smell	Por qué requiere escalamiento	A quién escalar
Toxic Leadership	El líder tiene poder formal sobre el equipo; la confrontación directa es riesgosa	CTO, VP Engineering, HR (con cuidado)
Organizational Silo (estructural)	Equipos separados por decisiones organizacionales	VP Engineering, Director de Producto
Unstable Team (política)	Las decisiones de staffing provienen de niveles superiores	CTO, Resource Manager
Budget Constraints	Problemas de-	CTO, VP con

Componente	Descripción
Hipótesis	Declaración falsificable que conecta una intervención con un resultado esperado. Formato: *Si [intervención], entonces [resultado], porque [mecanismo].*
Métricas	Indicadores cuantitativos definidos con baseline y objetivo (*target*).
Diseño	Definición de qué se hará, cuándo, quiénes participan y recursos requeridos (SSP).
Criterios de evaluación	Éxito: métricas alcanzan el *target* $\rightarrow$ institucionalizar. Parcial: mejora $\geq 50\,\%$ $\rightarrow$ iterar. Fracaso: sin mejora $\rightarrow$ pivotar. Inconcluso: datos insuficientes $\rightarrow$ extender experimento.
Documentación	Registro estructurado de la hipótesis, resultados obtenidos, aprendizajes y decisión final.

Tabla 11.132: Componentes de un experimento de mejora

No toda intervención requiere experimento formal. Se recomienda diseño experimental cuando: el contexto es único, la intervención es costosa (>8 SSP), el equipo es escéptico, o el impacto afecta múltiples equipos. Intervenciones simples y reversibles pueden implementarse directamente y ajustarse.

ANEXO K: PRINCIPIO DE NO-SOBRECARGA: INTEGRACIÓN SIN AGREGAR TIEMPO

I.3 K1. Principio de No-Sobrecarga: Integración sin agregar tiempo

Una preocupación frecuente ante cualquier extensión metodológica es el tiempo adicional que consume. Esta sección demuestra que SocialScrum, implementado correctamente, agrega una sobrecarga mínima: aproximadamente 45 minutos cada dos semanas, equivalente al 0.6 % del tiempo total del Sprint.
El principio fundamental es **integración, no adición**: los componentes sociales de SocialScrum se incorporan a los eventos Scrum existentes, no crean reuniones nuevas.

I.3.1 Sobrecarga temporal por evento

Evento	Scrum estándar	SocialScrum	**Sobrecarga** (redistribución interna)	% Aumento
Social Sprint Planning	4 horas	4 h 15 min	+15 min	+6 %
Social Daily Standup (x10)	150 min total	170 min total	+20 min	+13 %
Social Sprint Review	2 horas	2 h 10 min	+10 min	+8 %
Social Sprint Retrospective	1.5 horas	1.5 horas	+0 min	0 %
Total por Sprint	**10 horas**	**10 h 45 min**	**+45 min**	**+7.5 %**

Tabla 11.134: Sobrecarga temporal de SocialScrum por evento

I.3.1.1 Detalle por evento

Evento	Cambio en SocialScrum	Tiempo adicional
Social Sprint Planning	Tema 3: Riesgos sociales (Health Score, *community smells* activos e ítems sociales)	+15 min
Social Daily Standup (por día)	Pregunta final: "¿Hay algún impedimento social hoy?" (solo identificar, no resolver)	+2–3 min/día
Social Sprint Review	Sección dedicada a la evaluación de la dinámica social y la correlación entre Health Score y velocidad	+10 min
Social Sprint Retrospective	Redistribución del tiempo: inspección técnica (30 min), inspección social (45 min) y definición de acciones (15 min)	+0 min

Tabla 11.136: Detalle de cambios por evento

I.3.2 Sesiones adicionales y presupuesto

Algunos items sociales requieren sesiones fuera de eventos estándar (mediaciones 1:1, talleres de cohesión). Estas sesiones sí representan tiempo adicional, pero están acotadas:

- **Máximo 20 % de capacidad del equipo** para items sociales.
- **Máximo 2 horas de sesiones adicionales por semana** (fuera de eventos Scrum).

Optimización	Descripción	Ahorro
Daily asíncrono	Reemplazar el Daily síncrono por un formato escrito en Slack o Teams (campos: DONE, TODO, BLOCK, SOCIAL)	Elimina 15–18 min/día de reunión
Retros alternadas	Sprint impar: énfasis técnico (50/25 min). Sprint par: énfasis social (25/50 min)	Reduce la fatiga de retrospectiva
Social Guide rotativo	En equipos de 5–7 personas, rotar el rol cada Sprint. Dedicación aproximada: 10 % del tiempo cuando le corresponde	Distribuye la carga entre todos
Integración primero	Jerarquía de integración: Daily → Planning → Review → Retrospective. Solo crear una sesión adicional si no cabe en los eventos existentes	Minimiza sesiones adicionales

Tabla 11.138: Optimizaciones para reducir sobrecarga

I.3.4 Gestión de expectativas

I.3.4.1 Comunicación al equipo

Mensaje clave: "SocialScrum agrega 45 min cada 2 semanas a nuestras reuniones Scrum (4.5 min/día). No creamos reuniones nuevas; nos integramos en las existentes. Beneficio esperado: equipos con mejor comunicación entregan más a largo plazo."

I.3.4.2 Comunicación a stakeholders

Mensaje clave: "Implementamos SocialScrum para mejorar sostenibilidad del equipo. Verán un Health Score en cada Sprint Review. Sobrecarga: 0.6 % del tiempo. ROI: una renuncia evitada ahorra 3-6 meses de capacitación del reemplazo."

I.3.5 Retorno de inversión (ROI) de SocialScrum

El retorno de inversión se estima en dólares y pesos colombianos con conversión de $1 dólar es igual a $4.000 COP, en este escenario se hace un ejemplo mostrado en las tablas 11.140 y 11.142.

Inversión (anual)	Horas	Detalle	Costo (USD)	Costo (COP)
Sobrecarga de reuniones	19.5 h	45 min × 26 Sprints	$975	∼3 900 000 COP
Sesiones adicionales	130 h	∼5 h × 26 Sprints	$6 500	∼26 000 000 COP
Total	**150 h**	Costo anual estimado	**$7 500**	**∼30 000 000 COP**

Tabla 11.140: Inversión anual estimada en sobrecarga de SocialScrum

Retorno (anual)	Valor estimado
1 renuncia evitada	$50 000–$100 000 dólares (50–200 % del salario anual)

ANEXO L: EJEMPLOS DETALLADOS DE INTERVENCIONES

Este anexo presenta ejemplos detallados de intervenciones para diferentes categorías de *community smells*. Cada ejemplo incluye diagnóstico, estrategia paso a paso, métricas de éxito y resultados observados.

L.1 Intervención: *Radio Silence* (Smell de Comunicación)

```
=== INTERVENCIÓN: RADIO SILENCE ===
Equipo: Alpha (7 personas)
Severidad detectada: Moderada (afecta velocidad, no bloquea)

DIAGNÓSTICO:
- SNA muestra que María centraliza 70% de comunicación
- 3 DEVs reportan "enterarse tarde" de cambios de API
- En Daily, solo María y Pedro hablan regularmente
- Health Score "Comunicación" 4.5/10
ESTRATEGIA (5 SSP):
Semana 1:
- Workshop "Comunicación distribuida" (2h)
- Crear canal #alpha-actualizaciones para cambios técnicos
- Norma: quien cambia API pública, postea en canal

Semana 2:
- Rotación de "facilitador de Daily" (cada día alguien diferente)
- Pregunta explícita en Daily: "¿Hay algo que otros deban saber?"

MÉTRICAS DE ÉXITO:
- Mensajes en #alpha-actualizaciones: >= 5/semana
- Participación en Daily: todos hablan al menos 3x/semana
- Encuesta: "Me entero a tiempo" sube de 4/10 a 7/10
```

```
RESULTADO (Sprint +1):
- Mensajes en canal: 8/semana [OK]
- Participación Daily: 6 de 7 hablan regularmente [OK]
- Encuesta: 6.5/10 (mejora parcial, continuar)
```

L.2 Intervención: Truck Factor = 1 (Smell de Conocimiento)

```
=== INTERVENCIÓN: TRUCK FACTOR = 1 ===
Equipo: Beta (5 personas)
Componente crítico: Motor de reglas de negocio
Único experto: Carlos

DIAGNÓSTICO:
- Carlos escribió 95% del código del motor
- Cuando Carlos está de vacaciones, bugs en motor quedan sin resolver
- Otros devs "tienen miedo" de tocar ese código

ESTRATEGIA (8 SSP distribuidos en 3 Sprints):

Sprint 1 (3 SSP):
- Carlos documenta arquitectura en 3 páginas
- Carlos hace pair programming con Ana en 2 tareas del motor

Sprint 2 (3 SSP):
- Ana hace tarea sola en motor, Carlos solo revisa
- Carlos hace pair con Pedro en 1 tarea
- Sesión de 2h: "Cómo debuggear el motor"

Sprint 3 (2 SSP):
- Pedro hace tarea solo, Carlos revisa
- Documentar "troubleshooting guide" para problemas comunes
- Evaluar: ¿Ana y Pedro pueden resolver bugs nivel 1-2?

MÉTRICAS DE ÉXITO:
- Truck factor: de 1 a 3 (Carlos + Ana + Pedro)
- Tiempo de resolución de bugs cuando Carlos no está: < 48h
```

```
- Ana y Pedro reportan confianza >= 6/10 para trabajar en motor

RESULTADO:
- Sprint 3: Ana resolvió bug de producción sola [OK]
- Confianza: Ana 7/10, Pedro 5/10 (continuar con Pedro)
- Truck factor efectivo: 2.5 (Ana casi independiente, Pedro parcial)
```

L.3 Intervención: Conflicto No Resuelto (Smell Relacional)

```
=== INTERVENCIÓN: UNRESOLVED CONFLICT ===
Equipo: Gamma (6 personas)
Partes: Dev A (frontend) vs Dev B (backend)
Origen: Desacuerdo sobre diseño de API hace 2 meses

DIAGNÓSTICO:
- Code reviews entre A y B son hostiles
- Otros miembros evitan tareas que requieren A+B
- Daily tiene tensión cuando ambos reportan

ESTRATEGIA (5 SSP):

Día 1 - Sesión 1:1 con Dev A (1h):
- Escuchar su perspectiva sin juzgar
- Identificar: ¿qué necesita para mejorar la situación?
- A dice: "B nunca acepta mis sugerencias, siempre impone"

Día 2 - Sesión 1:1 con Dev B (1h):
- Escuchar su perspectiva sin juzgar
- B dice: "A no entiende las restricciones de backend"

Día 3 - Preparación:
- Identificar intereses comunes (ambos quieren producto exitoso)
- Preparar estructura para sesión conjunta

Día 4 - Sesión conjunta facilitada (2h):
- Reglas: hablar en primera persona, no interrumpir
- Cada uno expresa su perspectiva (10 min c/u)
```

```
- Identificar puntos de acuerdo
- Negociar: ¿cómo manejar desacuerdos futuros?
- Documentar acuerdos firmados por ambos

Día 15 - Seguimiento (30 min):
- ¿Se están respetando acuerdos?
- Ajustar si es necesario

ACUERDOS DOCUMENTADOS:
1. Desacuerdos técnicos se discuten en 1:1, no en PR comments
2. Si no hay acuerdo en 30 min, escalar a Tech Lead
3. Ambos se comprometen a asumir buena intención del otro

MÉTRICAS DE ÉXITO:
- Code reviews sin comentarios hostiles: 4 semanas consecutivas
- A y B aceptan trabajar en tarea conjunta sin resistencia
- Health Score "Respeto" sube de 4/10 a 6/10

RESULTADO (Sprint +2):
- 0 comentarios hostiles en PRs [OK]
- Aceptaron tarea conjunta en Sprint 16 [OK]
- Respeto: 6.5/10 [OK]
```

L.4 Intervención: Meeting Hell (Smell Organizacional)

```
=== INTERVENCIÓN: MEETING HELL ===
Equipo: Delta (8 personas)
Problema: Promedio de 25h/semana en reuniones por persona

DIAGNÓSTICO:
- Auditoría de calendarios muestra:
  - 8h/semana: reuniones de Scrum (normal)
  - 10h/semana: reuniones con partes interesadas (\textit{stakeholders}) (excesivo)
  - 7h/semana: reuniones "informativas" donde equipo es pasivo
- Foco del equipo: 4.2/10 (crítico)
- Velocity cayó 30% en últimos 3 Sprints

ESTRATEGIA (5 SSP):
```

```
Semana 1 - Documentación:
- Crear spreadsheet: reunión, convocador, frecuencia, asistentes
- Calcular costo: horas $\times$ personas $\times$ tarifa hora $\approx$ \$X/semana

Semana 2 - Propuesta:
- Clasificar reuniones: eliminar, reducir frecuencia, o delegar
- Propuesta: representante único para reuniones informativas
- Establecer "Focus Fridays": sin reuniones externas

Semana 3 - Negociación:
- Reunión con \textit{stakeholders} clave (SM lidera)
- Presentar datos de impacto en velocity
- Acordar: representante rota, resumen escrito después

Semana 4 - Implementación:
- Focus Fridays en efecto
- Representante único para 5 reuniones
- Monitorear calendarios

MÉTRICAS DE ÉXITO:
- Horas en reuniones: de 25h a 15h/semana
- Foco: de 4.2 a 6.0
- Velocity: recuperar al menos 50% de la caída

RESULTADO (Sprint +2):
- Reuniones: 16h/semana (36% reducción) [OK]
- Foco: 5.8/10 (mejora significativa) [OK]
- Velocity: +20% (recuperación parcial, continuar)
```

L.5 Experimento: Reducir Knowledge Islands

```
=== EXPERIMENTO: REDUCIR KNOWLEDGE ISLANDS ===

CONTEXTO:
Equipo Zeta tiene 3 subgrupos: Frontend, Backend, Data.
Casi no hay comunicación entre subgrupos.
Integración al final de Sprint genera conflictos.
Health Score "Compromiso": 5.5/10
```

HIPÓTESIS:
"Si implementamos code reviews cruzados obligatorios
(1 PR/semana revisado por miembro de otro subgrupo),
entonces el conocimiento compartido aumentará
y los conflictos de integración disminuirán,
porque la exposición temprana a código de otros
reduce sorpresas y builds empathy."

MÉTRICAS:

Métrica	Linea base	Target
PRs con reviewer cruzado	0/semana	5/semana
Conflictos de integración	4/Sprint	2/Sprint
"Entiendo código otros"	3/10	5/10
Health Score Compromiso	5.5	6.5

INTERVENCIÓN:
Semana 1:
- Workshop: "Por qué code reviews cruzados" (1h)
- Configurar GitHub: reviewer aleatorio de otro subgrupo
- Norma: al menos 1 PR cruzado por persona/semana

Semanas 2-4:
- Monitorear cumplimiento
- 1:1 con personas que no participan
- Celebrar en Daily cuando review cruzado genera insight

DURACIÓN: 2 Sprints

CRITERIOS DE EVALUACIÓN:
- Éxito: Conflictos <= 2, "Entiendo código" >= 5
- Parcial: Conflictos <= 3, alguna mejora en percepción
- Fracaso: Sin mejora o resistencia activa del equipo

RESULTADOS (Sprint +2):
- PRs cruzados: 6/semana [OK] (superó target)
- Conflictos: 3/Sprint (mejora, no alcanzó target)
- "Entiendo código": 4.5/10 (mejora parcial)
- Compromiso: 6.0/10 (mejora)

EVALUACIÓN: Éxito parcial

APRENDIZAJES:
1. Reviews cruzados toman más tiempo (factor 1.5x)
2. Backend → Frontend funciona mejor que inverso
3. Data team necesita onboarding adicional para participar

DECISIÓN: Iterar
- Reducir a 1 PR cruzado cada 2 semanas (más sostenible)
- Sesión de 1h para Data team sobre arquitectura general
- Re-evaluar en Sprint 20